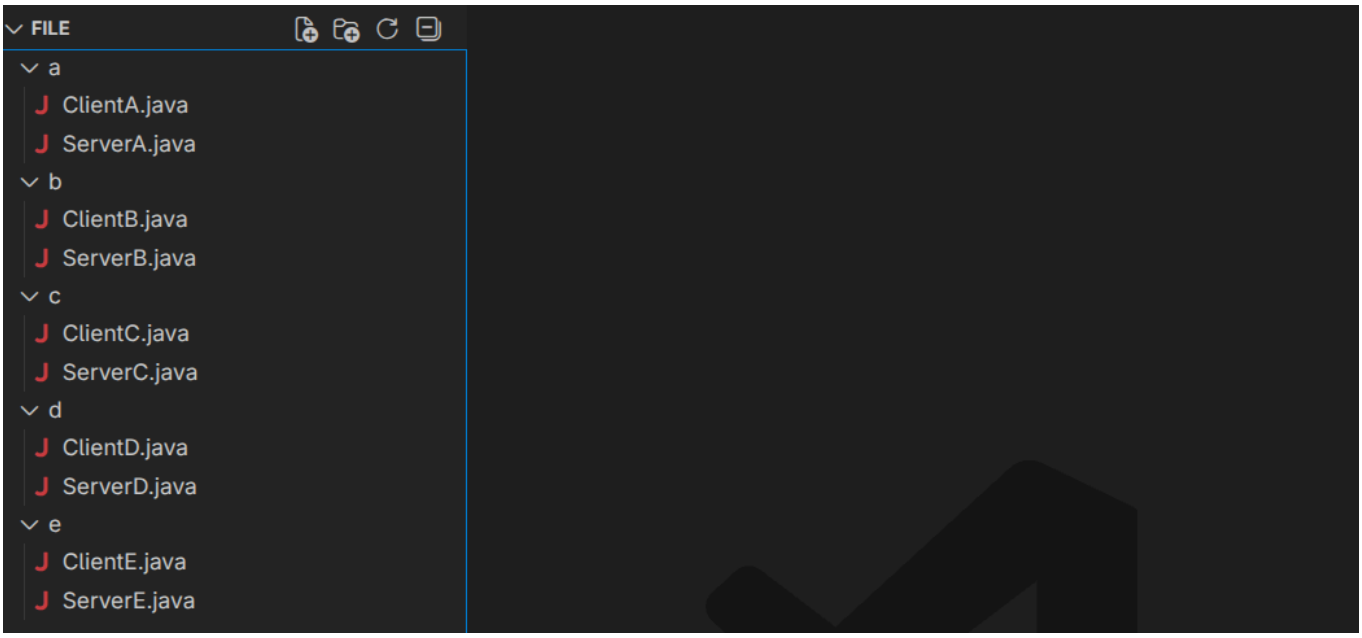
This lab is a little lighter, likely because we have an exam tomorrow. The only goal for this lab is to get practice with Sockets in Java.

We have a `file.zip` containing code for today's lab, so i'll unzip that in the folder for lab_5, and then jump into my ubuntu container set up with java. I open up the file in VSCode, and we can see it is divided into a few versions of a Client and Server pair, corresponding to steps in the lab.



Part a:

Compile part a files, run server, and run client in a separate terminal. Observe what happens.

Compiling:

```
📦[b@ubuntu a]$ javac *.java
📦[b@ubuntu a]$ ls
ClientA.class  ClientA.java  ServerA.class  ServerA.java
```

Because I'm in a container, I'm using tmux to run separate terminals for Server and Client.

Run Server:

```
📦[b@ubuntu a]$ java ServerA
Server started.
Listening on port: 38885
```

Run Client:

```
Exception in thread "main" java.net.ConnectException: Connection refused
        at java.base/sun.nio.ch.Net.connect0(Native Method)
        at java.base/sun.nio.ch.Net.connect(Net.java:601)
        at java.base/sun.nio.ch.Net.connect(Net.java:590)
        at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:583)
        at java.base/java.net.SocksSocketImpl.connect(SocksSocketImpl.java:327)
        at java.base/java.net.Socket.connect(Socket.java:751)
        at java.base/java.net.Socket.connect(Socket.java:686)
        at java.base/java.net.Socket.<init>(Socket.java:555)
        at java.base/java.net.Socket.<init>(Socket.java:324)
        at ClientA.main(ClientA.java:10)
```

Oops! Our connection was refused. Let's check our client code:

ClientA.java

```java
String host = "localhost";
int port = 12345;
```

There's the issue: Our client is attempting to connect to port 12345, and our server is listening on port 38885.

Let's change ClientA.java to reflect this.

ClientA.java

```java
String host = "localhost";
int port = 38885;
```

Now, I recompile ClientA.java, then rerun it.

Client:

```
📦[b@ubuntu a]$ javac ClientA.java
📦[b@ubuntu a]$ java ClientA
Received random number from server: 44
```

Server:

```
📦[b@ubuntu a]$ java ServerA
Server started.
Listening on port: 38885
Client connected.
Sent random number: 44
```

Much better! Our connection was accepted, and we can see on both the client and server end, a random number was sent successfully from server to client.

Part b:

Compile part b files, run server, run client.

Compile and start server:

```
📦[b@ubuntu b]$ javac *.java
📦[b@ubuntu b]$ java ServerB
Server started.
Listening on port: 51155
```

Run client:

```
📦[b@ubuntu b]$ java ClientB
Connected to server on port 51155
Enter a number or 'exit':
3
Server says: Server added 9, result = 12
exit
Server says: Goodbye!
Client closed.
```

Our client for part b is asked to submit a number, then the server will add a random number and return the sum.

Run the server and try to run 2 clients in separate terminals.

With the server running, I'll open the first instance of our Client:

Client 1:

```
📦[b@ubuntu b]$ java ClientB
Connected to server on port 51155
Enter a number or 'exit':

[0] 0:Client1* 1:Server  2:Client2-   "bluefin" 20:07 26-Feb-26
```

Then let's try to connect with the other, while the "Enter a number" I/O is hanging in Client1.

Client 2:

```
📦[b@ubuntu a]$ cd ../b                                    20:08:36 [8/8]
📦[b@ubuntu b]$ java ClientB
Connected to server on port 51155
Enter a number or 'exit':
```

Looks like both clients connected okay, let's try sending numbers from both.

Client1:

```
Enter a number or 'exit':
3
Server says: Server added 4, result = 7
```

On Client2, we are connected to the server, but no numbers are being sent to the client.

```
^C📦[b@ubuntu b]$ java ClientB
Connected to server on port 51155
Enter a number or 'exit':
1
exit
hellooooooo


[0] 0:Client1  1:Server- 2:Client2*   "bluefin" 20:12 26-Feb-26
```

We can also see there is no record of data from Client2 on the server end either, just interactions with Client1.

```
Server started.
Listening on port: 51155
Client connected.
Client sent: 3
Responded with result: 7
Client sent: 3
Responded with result: 44

[0] 0:Client1  1:Server* 2:Client2-   "bluefin" 20:13 26-Feb-26
```

Part c:

Read over code, compile part c files, run server

Upon reading the code for part c, it looks like the server has two I/O reads asking for two numbers to sum.

Let's compile and run:

```
📦[b@ubuntu b]$ cd ../c
📦[b@ubuntu c]$ javac *.java
📦[b@ubuntu c]$ java ServerC
Server listening on 127.0.0.1:51155

📦[b@ubuntu b]$ cd ../c
📦[b@ubuntu c]$ java ClientC
Connected to 127.0.0.1:51155
Enter first number (or 'exit'):
```

Now, we are instructed to start the server from part b:

```
📦[b@ubuntu a]$ cd ../b
📦[b@ubuntu b]$ java ServerB
Exception in thread "main" java.net.BindException: Address already in use
        at java.base/sun.nio.ch.Net.bind0(Native Method)
```

We get an error: `Address already in use` because this server is trying to use the same port ServerC is listening on.

Now, same thing with server from part a:

```
📦[b@ubuntu a]$ java ServerA
Server started.
Listening on port: 46799
```

No errors. ServerA has code to use a random available port:

ServerA.java

```java
// 0 = automatically assign random available port
ServerSocket serverSocket = new ServerSocket(0);
```

So we don't have to worry about port conflicts.

Try changing the port to 8080. Does it work?

I'll bring down ClientC and ServerC, then edit them to listen/connect on 8080.

ClientC:

```
📦[b@ubuntu c]$ java ClientC
Connected to 127.0.0.1:8080
Enter first number (or 'exit'): 2
Enter second number: 4
Server says: Sum(2+4) + random(36) = 42
Enter first number (or 'exit'):
```

ServerC:

```
📦[b@ubuntu c]$ javac *.java
📦[b@ubuntu c]$ java ServerC
Server listening on 127.0.0.1:8080
Client connected.
Received: 2, 4
Sent: Sum(2+4) + random(36) = 42
```

Works properly on both client and server end.

part d:

Read over the code, compile, run server, then run 4 separate clients.

Looks like the code for ServerD handles each client in a new thread using a a class called `ClientHandler` that implements `Runnable`.

Let's compile and get server going:

```
📦[b@ubuntu c]$ cd ../d
📦[b@ubuntu d]$ javac *.java
📦[b@ubuntu d]$ java ServerD
Server listening on 127.0.0.1:51155
```

In each client window, I've entered the following:

```
📦[b@ubuntu a]$ cd ../d
📦[b@ubuntu d]$ java ClientD
Connected to 127.0.0.1:51155
Enter two numbers separated by space (or 'exit'):
[0] < 2:Client2  3:Client3- 4:Client4*"bluefin" 20:37 26-Feb-26
```

Now, the server shows:

```
📦[b@ubuntu c]$ cd ../d
📦[b@ubuntu d]$ javac *.java
📦[b@ubuntu d]$ java ServerD
Server listening on 127.0.0.1:51155
New client connected: /127.0.0.1
New client connected: /127.0.0.1
New client connected: /127.0.0.1
New client connected: /127.0.0.1
[0] 0:Client1  1:Server* 2:Client2  3>"bluefin" 20:38 26-Feb-26
```

4 clients connected. Cool!

part e:

Read over the code, compare to part d, compile, run server, then run 4 clients.

ServerE uses a thread pool, which ServerD does not.

Let's compile and run server:

```
📦[b@ubuntu e]$ javac *.java
📦[b@ubuntu e]$ java ServerE
Server listening on 127.0.0.1:51155
Thread pool size: 3
```

Making note that the thread pool is only 3, and we are connecting 4 clients. I'm assuming one of our clients will be left out :(

In each Client:

```
📦[b@ubuntu a]$ cd ../e
📦[b@ubuntu e]$ java ClientE
Connected to 127.0.0.1:51155
Enter two numbers separated by space (or 'exit'):
```

We can see in ServerE, all clients are connected:

```
📦[b@ubuntu e]$ java ServerE
Server listening on 127.0.0.1:51155
Thread pool size: 3
Client connected: /127.0.0.1
Client connected: /127.0.0.1
Client connected: /127.0.0.1
Client connected: /127.0.0.1
```

Let's try entering numbers on all 4.

Client 1:

```
1 1
Server says: Thread pool-1-thread-1 → Sum(1+1) + random(99) = 101
```

Client 2:

```
Enter two numbers separated by space (or 'exit'):
2 2
Server says: Thread pool-1-thread-2 → Sum(2+2) + random(55) = 59
```

Client 3:

```
Connected to 127.0.0.1:51155
Enter two numbers separated by space (or 'exit'):
3 3
Server says: Thread pool-1-thread-3 → Sum(3+3) + random(79) = 85
```

Client 4:

```
📦[b@ubuntu a]$ cd ../e
📦[b@ubuntu e]$ java ClientE
Connected to 127.0.0.1:51155
Enter two numbers separated by space (or 'exit'):
4 4
uh oh
helloooooooooo

[0] < 2:Client2  3:Client3- 4:Client4*"bluefin" 20:48 26-Feb-26
```

We can see client 4 gets no response, because there are no threads left to run the computation. Even though the client connected, there isn't enough threads to share with it.

Bonus:

How would we get socket communication working across two different machines?

- We can run a client and server on both machines to listen and send data to eachother.

What are the potential issues that can arise from this communication?

- Dynamic IP address/port assignments could make this break frequently, and other people may be attempt to get into exposed ports.